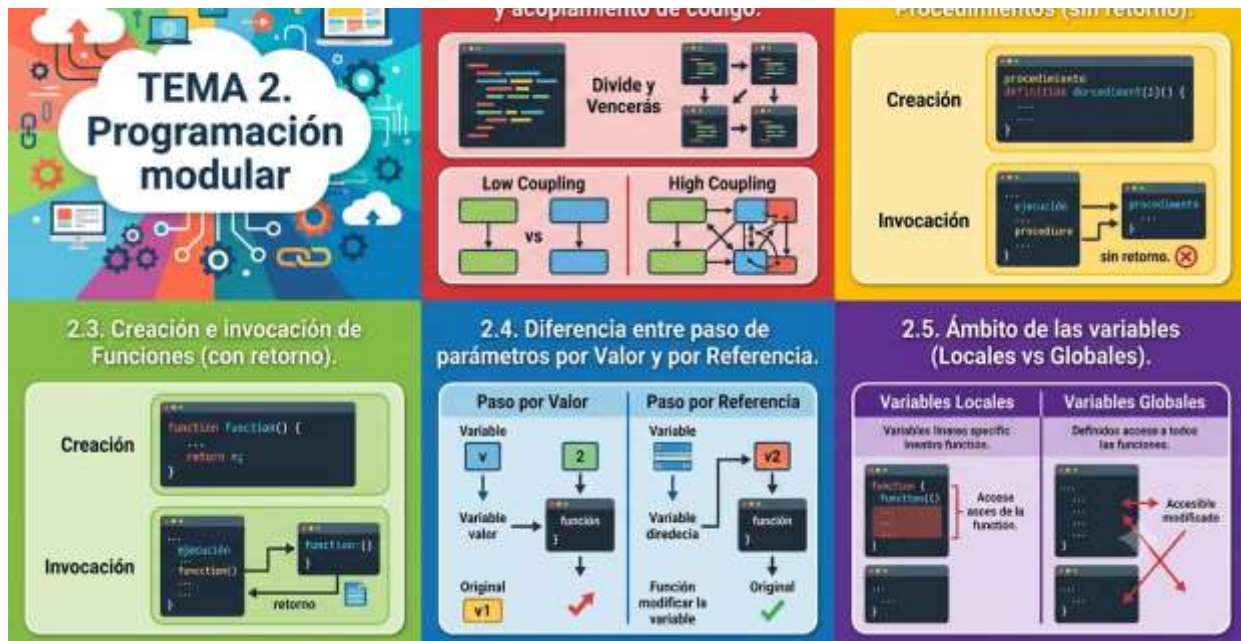


TEMA N° 2



TEMA 2 PROGRAMACIÓN MODULAR



"EN ESTE TEMA SERÁS CAPAZ DE..."

Como futuro Técnico en Sistemas Informáticos, te enfrentarás a retos tecnológicos cada vez más grandes. Construir software es como construir el nuevo módulo educativo de nuestro CEA: no se levanta todo el edificio de un solo golpe, sino bloque a bloque, aula por aula. La programación modular es exactamente eso: el arte de dividir sistemas masivos en piezas pequeñas, lógicas y manejables. Al finalizar esta unidad, tendrás el control total sobre la arquitectura de tu código.

Para lograrlo, nos enfocaremos en tres objetivos de aprendizaje fundamentales:

1. **Descomponer problemas complejos** utilizando la filosofía de "Divide y Vencerás", aislando tareas para construir sistemas más limpios y con bajo acoplamiento.
2. **Diseñar e invocar procedimientos y funciones** que agrupen lógica de programación reutilizable, evitando la duplicación de código en tus proyectos.



3. **Administrar eficientemente la memoria y el flujo de datos** comprendiendo a la perfección el ámbito de las variables (locales y globales) y el paso de parámetros (por valor y por referencia).



PRÁCTICA



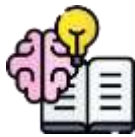
Pregunta detonante: *¿Alguna vez has intentado resolver un problema tan grande que, a mitad de camino, te sentiste completamente perdido entre tantas instrucciones y no sabías por dónde empezar ni cómo retroceder?*

El Reto Comunitario en Pailón:

Imagina que las autoridades del municipio de Pailón y la dirección del CEA Herman Ortiz Camargo te han encomendado crear el "Sistema Integrado de Gestión Pailonense". Este software debe hacer de todo: registrar a los nuevos estudiantes en nuestro edificio del Barrio Saó, organizar los horarios de los campeonatos deportivos en el coliseo municipal del Barrio 27 de Mayo, gestionar los espacios de los vendedores en el comercio formal del Barrio Central Fátima y, además, coordinar el mantenimiento de los parques infantiles en el Barrio Residencial Norte y el Barrio San Expedito.

En tu primer intento, abres tu editor de código y empiezas a escribir. Escribes la lógica de inscripción de estudiantes, y justo debajo, en el mismo archivo gigante, pones los cálculos de los impuestos de los gremiales, seguidos inmediatamente por el registro de los equipos de fútbol. De pronto, tienes un archivo con miles de líneas de código.

Un día, te piden cambiar el costo de la cancha en el Barrio 27 de Mayo. Al intentar hacer este pequeño cambio en tu inmenso archivo, accidentalmente borras una línea, y de repente, ¡el sistema de inscripciones del CEA en el Barrio Saó deja de funcionar! Todo está mezclado, todo depende de todo. El sistema es un caos y arreglar un error genera tres errores nuevos. ¿Cómo rediseñamos esta solución tecnológica para que cada sector del municipio funcione de forma independiente, pero integrada, sin que un cambio en los parques infantiles afecte al módulo educativo?



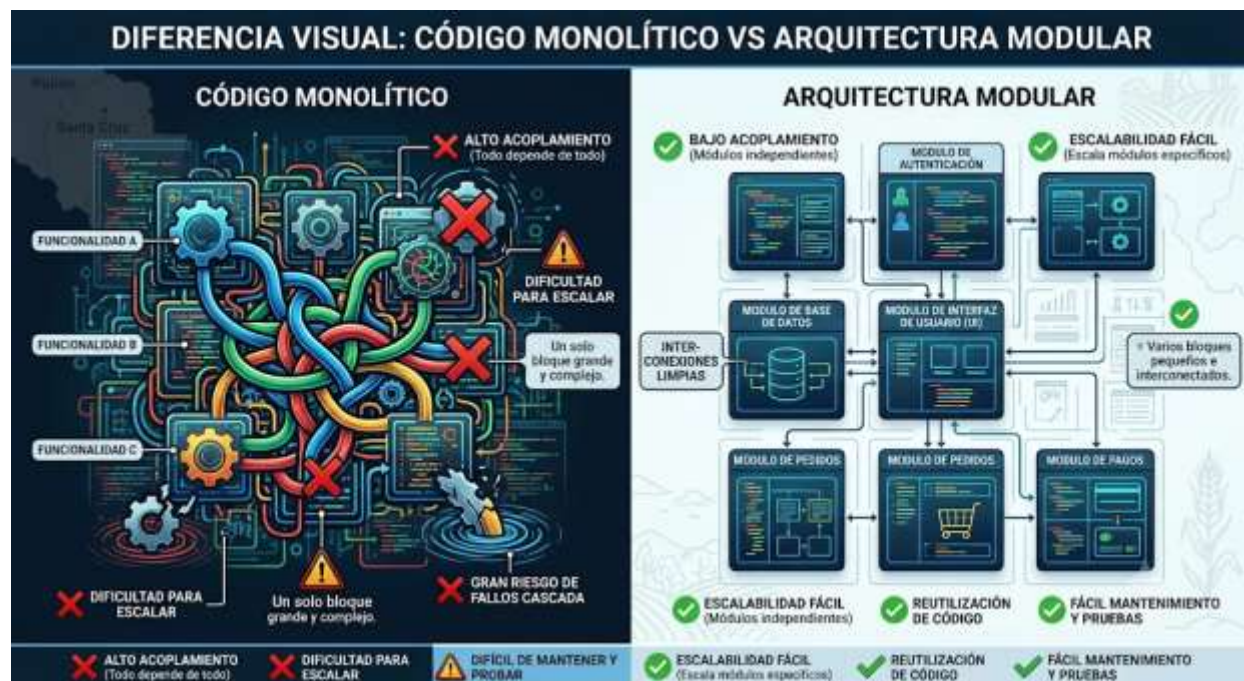
TEORÍA



2.1. CONCEPTO DE "DIVIDE Y VENCERÁS" Y ACOPLAMIENTO DE CÓDIGO

El principio de "Divide y Vencerás" (del latín *divide et impera*) es una estrategia fundamental en la informática. Como Técnico en Sistemas Informáticos, debes entender que ningún problema complejo debe abordarse en su totalidad. Se debe fragmentar en subproblemas más pequeños, simples y manejables (módulos).

En el contexto de la programación, un **módulo** es un fragmento de código que agrupa tareas relacionadas. En lugar de tener un archivo gigante (código espagueti), tienes piezas de Lego. Si el registro de estudiantes del CEA falla, solo revisas el "módulo de estudiantes", sin tocar el "módulo de deportes" del Barrio 24 de Septiembre.



Un concepto crítico aquí es el **acoplamiento**. El acoplamiento mide qué tan dependiente es un módulo de otro.

1. **Alto Acoplamiento (Malo):** Si cambias algo en la lógica del Barrio Fátima y el módulo del Barrio María Belén se rompe, tu código está altamente acoplado.



2. **Bajo Acoplamiento (Bueno):** Es el objetivo de la programación modular. Los módulos deben ser lo más independientes posible. Solo se comunican a través de canales estrictos y bien definidos.

2.2. CREACIÓN E INVOCACIÓN DE PROCEDIMIENTOS (SIN RETORNO)

Un **procedimiento** es un bloque de código empaquetado que realiza una acción específica, pero **no devuelve ningún dato** al programa principal una vez que termina. Piensa en un procedimiento como una orden directa que das para que algo suceda, como imprimir un recibo o mostrar un mensaje en pantalla. En muchos lenguajes (como Python), tanto los procedimientos como las funciones se definen igual, pero el procedimiento no usa la palabra clave `return`.

Caso práctico:

Necesitamos mostrar un mensaje de bienvenida estandarizado en la pantalla de los tótems digitales ubicados en la estación de tren del Barrio 24 de Septiembre. En lugar de escribir el mensaje repetidamente, creamos un procedimiento.

Python

```
# Definición del procedimiento usando la palabra reservada 'def'
def mostrar_bienvenida_estacion():    # Esta línea imprime el primer mensaje en
    print("
---
---
---
---
---
---
---
---
---
---
----")    # Esta línea muestra el mensaje principal de bienvenida
print(";Bienvenidos a La Estación del Barrio 24 de Septiembre!")    # Esta línea
añade información adicional    print("
Por favor, tenga su boleto a mano.")    # Esta línea cierra el bloque visual del
```



```
mensaje    print("
---
---
---
---
---
---
---
---
---
---
---
---
---
----")
# Invocación (llamada) del procedimiento en el programa
principalmostrar_bienvenida_estacion() # Si necesitamos el mensaje 100 veces,
solo llamamos la función 100 veces
# y no copiamos las instrucciones una y otra vez.
```

2.3. CREACIÓN E INVOCACIÓN DE FUNCIONES (CON RETORNO)

A diferencia de un procedimiento, una **función** realiza un cálculo o procesamiento interno y **devuelve un resultado** (un valor) al lugar donde fue invocada. Utiliza la instrucción `return`. Es como enviar una solicitud a la alcaldía: entregas documentos (datos), ellos los procesan, y te entregan un certificado (resultado).



**Caso práctico:**

El coliseo municipal en el Barrio 27 de Mayo alquila su cancha por horas. Necesitamos una función que reciba la cantidad de horas y el precio por hora, y nos devuelva el total a pagar.

Python

```
# Definición de la función con dos parámetros: horas y precio_hora
def calcular_alquiler_coliseo(horas, precio_hora):
    # Calculamos el costo multiplicando los parámetros
    costo_total = horas * precio_hora
    # Retornamos (devolvemos) el resultado al programa principal
    return costo_total

# INVOCACIÓN DE LA FUNCIÓN
# Llamamos a la función enviando 3 horas a 50 bolivianos cada una
total_a_cobrar = calcular_alquiler_coliseo(3, 50)

# Imprimimos el resultado devuelto por la función
print("El total a cobrar por el coliseo es: Bs.", total_a_cobrar)
```

2.4. DIFERENCIA ENTRE PASO DE PARÁMETROS POR VALOR Y POR REFERENCIA

Cuando envías información (parámetros) a una función o procedimiento, los lenguajes de programación lo manejan de dos maneras distintas, lo cual es vital que todo Técnico en Sistemas Informáticos entienda para evitar pérdida de datos:

1. **Paso por Valor:** Se envía una **copia** del dato original. Si la función modifica este dato internamente, el original queda intacto. Es como si le dieras a un turista una fotocopia del mapa del Barrio San José. Si el turista raya la fotocopia, tu mapa original sigue perfecto. *Se usa comúnmente con tipos de datos simples (números, textos, booleanos).*
2. **Paso por Referencia:** Se envía la **dirección de memoria** del dato original. No hay copia. Si la función lo modifica, el dato original cambia en todo el programa. Es como darle a alguien la llave original de un aula en el CEA Herman Ortiz Camargo; si esa persona entra y pinta la pared, cuando tú entres después, la pared estará pintada. *Se usa comúnmente con tipos de datos complejos (listas, arreglos, objetos).*

Caso práctico en código:

Veamos cómo una lista (paso por referencia) es alterada.

Python



```
# Función que recibe una lista de barrios (Paso por referencia)
def agregar_barrio(lista_barrios):    # Añadimos un nuevo barrio a la lista que
    recibimos    lista_barrios.append("
Ovidio Barberi")    # Imprimimos la lista dentro de la función    print("
Dentro de la función:", lista_barrios)
# Programa principal
# Creamos nuestra lista original en la memoria mis_barrios = ["
Barrio Saó", "
San Antonio"]
# Invocamos la función y le pasamos nuestra lista
original agregar_barrio(mis_barrios)
# Imprimimos la lista original DESPUÉS de la función
# NOTA: La lista original ha sido modificada permanentemente print("
Fuera de la función:", mis_barrios)
```

2.5. ÁMBITO DE LAS VARIABLES (LOCALES VS GLOBALES)

El ámbito (o *scope*) determina en qué partes de tu programa "existe" y es accesible una variable.

1. **Variables Globales:** Se declaran en el programa principal, fuera de cualquier función. Son accesibles desde cualquier parte del código. Son como la plaza central en Pailón: cualquier ciudadano de cualquier barrio puede verla y acceder a ella.
2. **Variables Locales:** Se declaran *dentro* de una función específica. Solo existen allí adentro y mueren cuando la función termina. Son como los equipos de computación dentro de un laboratorio específico en nuestro módulo del CEA; alguien que está caminando por la calle fuera del edificio no puede usarlos ni verlos.





Python

```
# VARIABLE GLOBAL: Declarada fuera de toda funciónimpuesto_municipal = 0.15 def
calcular_pago_vendedor_fatima(ventas): # VARIABLE LOCAL: '
descuento
' solo existe dentro de esta función    descuento = 20    # Podemos usar la
variable global '
impuesto_municipal
' aquí adentro    pago_final = (ventas - descuento) + (ventas *
impuesto_municipal)    return
pago_finalprint(calcular_pago_vendedor_fatima(500))
# Si intentamos imprimir '
descuento
' aquí afuera, EL PROGRAMA DARÁ ERROR
# print(descuento) # ERROR: NameError: name '
descuento
' is not defined
```

2.6. MODULARIDAD EN LA ERA DE LA INTELIGENCIA ARTIFICIAL

Hoy en día, las herramientas de Inteligencia Artificial como GitHub Copilot, Gemini o ChatGPT son asistentes diarios para un Técnico. Sin embargo, estas IA funcionan de manera mucho más precisa cuando trabajan con código modular. Si le pides a una IA "escribe un sistema entero de 10,000 líneas", probablemente cometa muchos errores de lógica (alucinaciones). Pero si aplicas el "Divide y Vencerás" y le pides: "Escribe una función independiente que calcule el área de los parques en el Barrio Las Misiones y devuelva un número flotante", la IA generará un código casi perfecto. La programación modular no es solo para humanos, es el estándar para interactuar eficazmente con la IA generativa.

2.7. MICROSERVICIOS: LA EVOLUCIÓN DE LA PROGRAMACIÓN MODULAR

La programación modular tradicional agrupa funciones dentro de un mismo programa y servidor. Sin embargo, la tendencia moderna lleva esto al extremo con la **Arquitectura de Microservicios**. En este enfoque, los módulos no solo están en archivos separados, sino que son programas completamente independientes, que incluso pueden estar en servidores distintos o escritos en lenguajes diferentes, comunicándose a través de la web (APIs). Empresas globales operan así. En el futuro, el sistema de Pailón podría tener el módulo del CEA en un servidor escrito en Python,



y el módulo del coliseo en la nube usando Node.js, funcionando en perfecta sincronía gracias a la modularidad escalada.

2.8. MANTENIBILIDAD Y CÓDIGO LIMPIO (CLEAN CODE)

Crear módulos no consiste solo en cortar el código al azar. Existe la filosofía de *Clean Code* (Código Limpio), la cual dicta que cada función o procedimiento debe tener **una sola responsabilidad** (Principio de Responsabilidad Única). Si tienes una función llamada `calcular_notas_y_guardar_en_base_de_datos_e_imprimir()`, estás violando este principio. Deberían ser tres funciones distintas. Un código modular bien escrito permite la refactorización continua: mejorar el código interno sin alterar su funcionamiento externo, asegurando que el software que construyamos en Pailón sea mantenible durante décadas.

CIERRE TEÓRICO OBLIGATORIO

Mitos sobre la Programación Modular	Realidad Tecnológica
"Escribir todo en un solo archivo es más rápido y fácil."	Solo es rápido los primeros 5 minutos. En cuanto el proyecto crece un poco, encontrar errores o añadir nuevas funciones en un código monolítico se vuelve una pesadilla inmanejable.
"Hacer funciones hace que la computadora procese el código más lento."	El impacto en velocidad de llamar a una función es imperceptible (microsegundos). El beneficio en claridad, seguridad y prevención de errores críticos compensa infinitamente este mito anticuado.

□ Glosario de Términos:

1. **Modularidad:** Técnica de diseño de software que divide un sistema complejo en componentes o módulos más pequeños, independientes y manejables.
2. **Procedimiento:** Subrutina de código que ejecuta una serie de instrucciones secuenciales sin devolver un valor final al sistema principal.
3. **Función:** Bloque de código reutilizable diseñado para procesar datos de entrada (parámetros) y retornar un resultado procesado.



4. **Parámetro:** Variable especial utilizada para recibir datos de entrada en una función o procedimiento en el momento de su invocación.
5. **Refactorización:** Proceso de reestructurar y mejorar el código fuente existente sin cambiar su comportamiento externo, para hacerlo más legible y modular.

□ **Ponte a Prueba:**

1. ¿Cuál es la principal ventaja de aplicar el principio de "Divide y Vencerás" en programación?
 1. a) Hace que la computadora gaste más energía.
 2. b) Reduce la complejidad y facilita el mantenimiento aislando problemas.
 3. c) Convierte automáticamente variables locales en globales.
2. Si enviamos una variable de tipo Lista a una función y su contenido se altera dentro de ella afectando al programa principal, ¿qué tipo de paso de parámetros ocurrió?
 1. a) Por Valor.
 2. b) Por Retorno.
 3. c) Por Referencia.
3. Una variable declarada estrictamente dentro del procedimiento calcular_notas() del CEA, no puede ser leída desde fuera. ¿A qué concepto nos referimos?
 1. a) Ámbito Local.
 2. b) Alto Acoplamiento.
 3. c) Ámbito Global.



VALORACIÓN



Más allá de la sintaxis y la lógica, la programación modular tiene un impacto profundo en nuestro entorno social y ambiental. Pensemos en el *e-waste* (basura electrónica) y la huella de carbono. Un código no optimizado, monolítico y enredado consume exponencialmente más memoria



RAM y ciclos de procesamiento (CPU). Cuando los sistemas del municipio de Pailón se ejecutan con este tipo de código ineficiente en los servidores, consumen más energía eléctrica y obligan a reemplazar el hardware (computadoras, servidores) en menor tiempo porque "el sistema se volvió pesado". Escribir código modular, limpio y bien estructurado es una forma de **Computación Verde (Green Computing)**; optimiza el consumo energético de las máquinas.

A nivel social y ético, la modularidad garantiza privacidad y seguridad. Si el sistema de "Gestión Pailonense" fuera monolítico y un pirata informático encontrara una vulnerabilidad en la sección de reservas de los campos deportivos del Barrio 24 de Septiembre, podría acceder a toda la base de datos de los estudiantes del CEA Herman Ortiz Camargo. Al mantener sistemas modulares y de bajo acoplamiento, establecemos cortafuegos arquitectónicos. Si un módulo se ve comprometido, los demás módulos y la información privada de nuestra comunidad chiquitana permanecen aislados y protegidos. Un Técnico responsable programa pensando en la seguridad de su gente.



PRODUCCIÓN



Reto Final: Laboratorio "Gestión Modular Comunitaria"

Ha llegado el momento de aplicar tus conocimientos en la computadora. Vamos a construir un prototipo en consola para la alcaldía usando modularidad. Abre tu entorno de desarrollo (ej. VS Code).

Paso 1: Crear el módulo de Bienvenida (Procedimiento)

1. Define un procedimiento llamado `iniciar_sistema()`.
2. Dentro, imprime mensajes de bienvenida creativos para el "Sistema de Gestión de Pailón".
3. Invócalo al final de tu script para probar que funciona.

Paso 2: Crear el módulo del Comercio Formal Fátima (Función con Valor)

1. Define una función `calcular_impuesto_gremial(ingreso_diario)`.
2. Dentro, calcula el impuesto (asume un 5% de impuesto local).



3. Utiliza una variable local para el porcentaje.
4. Retorna el valor del impuesto y muéstralo en pantalla llamando a la función.

Paso 3: Crear el módulo de Parques del Barrio Residencial Norte (Paso por Referencia)

1. Crea una lista global llamada `parques_necesitan_mantenimiento` = ["Parque Sur", "Parque Central"].
2. Crea una función llamada `reportar_parque_dañado(lista, nombre_parque)`.
3. Dentro de la función, utiliza `.append()` para añadir `nombre_parque` a la lista.
4. En tu programa principal, llama a la función pasándole tu lista y el nombre "Parque Norte". Imprime la lista original para verificar que el nuevo parque ha sido agregado exitosamente mediante paso por referencia.

Paso 4: Documentación y Comentarios

1. Asegúrate de agregar comentarios línea por línea a todo tu código, demostrando que comprendes qué hace cada fragmento y delimitando claramente dónde empieza una función y dónde el programa principal.

5. RECURSOS ADICIONALES

Para consolidar tu aprendizaje y llevar tus habilidades modulares al siguiente nivel, explora los siguientes recursos oficiales y comunitarios:

1. **MDN Web Docs - Funciones:** Una de las documentaciones más prestigiosas y didácticas del mundo. Aunque muy enfocada en JavaScript, sus explicaciones sobre modularidad y ámbito de variables son aplicables universalmente. (Busca: *MDN Functions guide*).
2. **Documentación Oficial de Python (Tutorial de Estructuras y Módulos):** La fuente primaria para entender cómo Python maneja la creación de funciones (`def`), el retorno de valores y la organización de archivos. Ideal para la práctica directa. (Busca: docs.python.org/es/tutorial/modules.html).
3. **Repositorios de Clean Code en GitHub:** Existen decenas de guías gratuitas y traducidas al español en GitHub bajo el término "Clean Code". Estudiar estos repositorios te enseñará



las mejores prácticas profesionales de la industria para nombrar tus variables, mantener tus funciones pequeñas y tu código altamente mantenible. (Busca: *Clean code español GitHub*).